MASTER'S PROJECT BRIEF

A wallet that shows its work.

Designing a transparent self-sovereign identity wallet — with a local VON ledger, a hierarchical broker, and a push channel that only wakes the wallets that need to listen.

Architecture · DIDComm protocols · Local VON network · Hierarchical broker · Implementation roadmap · Scientific contribution

Contents

1	Why this matters	3
2	The core idea: two windows of transparency	4
3	Wallet architecture	6
4	Talking to the world: DIDComm v2	8
5	The verifiable data registry: VON network	10
6	The push channel: watcher and broker	12
7	Hierarchical broker: the scientific core	14
8	Topics, events, and routing	16
9	Technology stack	18
10	Implementation roadmap	20
11	Scientific contribution and evaluation	22
12	Risks and mitigations	24
13	Closing thoughts	25

WHY THIS MATTERS

01 Trust is built by showing your work

Most identity wallets ask the user to approve or deny — without telling them what is about to happen, or what just did. The cryptography happens in silence; the ledger interactions happen in silence; the consent decision becomes a single tap on a button that hides a dozen privacy-relevant choices.

> *That is not transparency. That is a button.*

This project takes a different position. A self-sovereign identity wallet should let the user see three things: **what is being asked of them before they agree**, **what is happening behind the scenes while it happens**, and **what was done after it is over**. The wallet earns trust by showing its work — not by asking the user to take it on faith.

What this project delivers

A working transparent SSI wallet, running against a local VON Indy ledger, connected through a hierarchical message broker, with two distinct transparency channels surfaced in the UI and persisted in a tamper-evident audit log.

- A holder agent that speaks DIDComm v2 with issuers, verifiers, and the broker.
- A local VON network as the verifiable data registry — four Indy nodes plus a ledger browser, all in Docker.
- A ledger watcher that subscribes to ledger events and republishes them as DIDComm push notifications.
- A hierarchical broker — root broker fans out to regional brokers, regional brokers route to wallets — with topic-based subscriptions so only the wallets that care about a particular DID receive the notification.
- Two transparency layers in the wallet UI: an on-request consent surface, and a behind-the-scenes live activity log.
- A hash-chained audit log that the user can export as cryptographic proof of what their wallet actually did.

The framing question

Can a wallet that exposes more of its internal operation actually *increase* user trust, rather than overwhelming them? This project's answer is yes — but only if the two transparency channels are designed for two different audiences and two different moments. The on-request channel is for the deciding user. The behind-the-scenes channel is for the auditing user, the regulator, and the researcher.

02

THE CORE IDEA

Two windows of transparency

The wallet has two distinct transparency surfaces. They are not styling variants of the same view — they answer different questions, surface at different moments, and serve different audiences.

On-request transparency

This surface appears at the moment of a decision: just before the user agrees to issue a credential to themselves, or share a credential with a verifier. It answers: *what is about to happen?*

- **Who is asking** — the verifier's DID, their human-readable name, their position in any trust registry the user subscribes to.
- **What is being asked for** — the exact attributes from the exact credentials, mapped to plain language so the user can check whether 'date of birth' matches their expectation of 'over 18'.
- **What it will be used for** — the verifier's stated purpose, surfaced from the presentation request's purpose binding.
- **How long it will be kept** — the verifier's retention policy, where supplied.

Behind-the-scenes transparency

This surface appears *during* a flow and persists after it. It answers: *what is the wallet actually doing right now, cryptographically?* It is the live narration of every operation that crosses a layer boundary inside the wallet, rendered in plain language and committed to a tamper-evident log.

- **Resolved** the verifier's DID document from did:web at example.com.
- **Fetched** the schema for university-diploma v2 from the Indy ledger.
- **Checked** the revocation status for credential index 42 (clean).
- **Generated** a selective-disclosure proof for {givenName, degree} — birthdate not revealed.
- **Signed** the presentation with the device-bound key in the secure enclave.
- **Recorded** log entry #1247, hash linked to entry #1246, anchored to ledger transaction 8742.

> **Why two channels and not one**
>
> On-request information helps the deciding user; behind-the-scenes information would overwhelm them at decision time. Behind-the-scenes information helps the auditing user, but the auditing user is rarely the same person as the deciding user, and rarely needs the information at the same moment. Collapsing the two into one channel produces the worst of both worlds: too much for decisions, too little for audits.

WALLET ARCHITECTURE

03 Four layers, talking only to neighbours

The wallet is structured as four horizontal layers. Each layer talks only to its immediate neighbours, which means every cross-layer call is an interception point where the wallet can emit a structured, human-readable event into the behind-the-scenes log. This is what makes meaningful transparency mechanically possible.

Layer	Responsibility	Examples
1. UX	Surfaces the inbox, consent prompts, and activity log.	credential cards, disclosure picker, audit history view
2. Core logic	Orchestrates flows. Decides policy. Builds presentations.	presentation exchange engine, consent engine, trust resolver
3. Crypto and protocol	DIDs, credential formats, proofs, transport.	DID resolver, proof engine (SD-JWT-VC, BBS+, AnonCreds), DIDComm v2
4. Secure storage	Keys, credentials, and the audit log — all encrypted at rest.	secure enclave / HSM, encrypted SQLCipher store, hash-chained log

The four wallet layers and their responsibilities.

Why strict layering

A wallet that lets the UX layer reach directly into the key vault — to sign something on behalf of the user — is a wallet that cannot reliably tell the user what it just signed. By forcing every signature request to travel UX → core logic → crypto → storage, the wallet creates a controlled chain where each step is logged. The cost is some indirection. The benefit is that transparency is no longer a UI feature bolted on top — it is a property of the architecture.

The transparency channels run alongside

The two transparency surfaces are not a fifth layer. They are observers. They subscribe to events emitted at the layer boundaries and render them in their respective views. The on-request surface filters for events tagged `consent-requested`; the behind-the-scenes surface shows everything except sensitive payloads.

Design principle

Strict layering plus event emission at every boundary. Every cross-layer call produces exactly one log entry. The audit log is therefore complete by construction, not by careful instrumentation.

TALKING TO THE WORLD

04 Three conversations, one encrypted line

All wallet-to-outside-world communication uses DIDComm v2. End-to-end encrypted, mutually authenticated by DIDs, transport-agnostic. The wallet runs as a DIDComm agent — it has a DID, a service endpoint (typically a mediator), and a set of protocol handlers that dispatch incoming messages by type.

Three protocols on one transport

issue-credential v2

An issuer offers the wallet a verifiable credential. The wallet can accept, propose alternative attributes, or decline. On acceptance, the issuer signs and sends, and the wallet stores. The exchange is governed by Aries RFC 0453.

present-proof v3

A verifier sends a presentation request describing the attributes and predicates they need to verify. The wallet's presentation exchange engine matches the request against held credentials, surfaces a disclosure picker to the user, generates a proof (selective disclosure where supported), and sends the presentation back. The exchange is governed by Aries RFC 0454.

vdr-notify (custom protocol)

This is the new protocol contribution of the project. The watcher service publishes ledger events on the broker as DIDComm-formatted messages. The broker forwards them down the topic hierarchy until they reach wallets whose subscriptions match. Each message carries the event type, the affected DID or credential, the originating ledger transaction reference, and a watcher signature.

> **Why use DIDComm for the push channel**
>
> Reusing DIDComm for the watcher-to-wallet leg means the wallet does not need a second authentication mechanism. The watcher's DID is the only credential needed to verify a notification's origin. Encryption, replay protection, and message framing come for free from the existing protocol.

The mediator

Wallets running on phones are intermittently online. The mediator solves this. A mediator is a long-running DIDComm endpoint that holds messages on the wallet's behalf. The wallet registers with the mediator at onboarding; thereafter, all external parties — issuer, verifier, broker — send to the mediator's endpoint, addressed to the wallet's DID. When the wallet next comes online, it picks up queued messages using the messagepickup protocol.

Wakeup

On phones, a separate platform-specific channel (APNs on iOS, FCM on Android) is used to wake the wallet so it can fetch from the mediator. The wakeup notification carries no DIDComm payload — only a hint that something is waiting. This keeps mobile platforms' notification routing infrastructure separate from the trust-bearing DIDComm layer.

THE VERIFIABLE DATA REGISTRY

05 Building on a local VON network

VON — Verifiable Organisations Network — is the open Hyperledger Indy testbed maintained by the British Columbia government. It is the standard local Indy development environment in the SSI community. For a master's project it is ideal: fully under our control, reproducible, and demonstrable in real time during a defence.

What VON gives us

- Four Indy ledger nodes — one steward, three validators — running in Docker containers.
- A genesis transaction file we can hand to any agent or watcher to connect to the pool.
- A ledger browser at `localhost:9000` that lets us inspect every transaction live.
- Tools to write DIDs, schemas, credential definitions, and revocation registries during demos.
- A complete reset capability — we can wipe the ledger and replay scenarios as often as needed.

Bringing it up

From a clone of the bcgov/von-network repository:

```
git clone https://github.com/bcgov/von-network.git
cd von-network
./manage build
./manage start --logs
```

The pool is then alive at `localhost:9701-9708` and the browser at `localhost:9000`.

What we will write to it

- **Steward DIDs** — for the test issuers (a fictional government, university, and clinic).
- **Schemas** — for example a passport schema, a diploma schema, a vaccination schema.
- **Credential definitions** — bound to schemas, owned by issuers.
- **Revocation registries** — backed by cryptographic accumulators, used to revoke individual credential indices.

> **About AnonCreds**
>
> Indy uses AnonCreds as its credential format — different from W3C VC Data Model JSON-LD credentials and different from SD-JWT-VC. AnonCreds has built-in selective disclosure and predicate proofs (e.g. age > 18) via CL-signatures and zero-knowledge proofs. The trade-off: it is Indy-specific. The wallet supports both AnonCreds (for credentials issued via VON) and SD-JWT-VC (for credentials issued via did:web issuers).

Why a local ledger and not a public one

A public Indy ledger such as Sovrin MainNet would require write fees, governance approval for stewards, and would couple the project to a real network's uptime. A local VON network has none of these constraints, gives us deterministic state for experiments, and lets us measure throughput and latency without confounders from a shared production environment.

THE PUSH CHANNEL

06 Watcher and broker

Indy ledgers do not push. They are pull-based by design — clients query for the latest transactions or for the state of a specific object. To move from pull to push, we introduce a watcher service that polls the ledger on the wallets' behalf and republishes events on a broker. Wallets subscribe to the broker and receive only the events that match their interests.

The watcher

The watcher is a long-running service. It has its own DID and signs every event it publishes. Its responsibilities are:

- **Poll the Indy pool** using `indy-vdr`, maintaining a cursor on the latest sequence number.
- **Diff new transactions** against the watcher's local cache of state.
- **Classify each transaction** by type — NYM (DID), SCHEMA, CRED_DEF, REVOC_REG_ENTRY, ATTRIB — and convert to a semantic event.
- **Resolve revocation accumulator deltas** back to individual credential indices, using the issuer's tails file.
- **Build a topic** following the schema `ssi.{realm}.{type}.{did}`.
- **Sign and publish** the event message to the broker.

> **The hard part: revocation**
>
> Indy uses cryptographic accumulators for revocation. When an issuer revokes a credential, the on-chain transaction is an updated accumulator value — not *which credential was revoked*. To recover that information, the watcher must hold the issuer's tails file (a binary that contains all credential indices for a registry) and compute the diff between the old and new accumulator. This is a non-trivial implementation detail and a genuine engineering contribution of the project.

The broker

The broker accepts published events from the watcher and routes them to subscribed wallets. It is the structural piece that makes scaling possible: a single watcher can publish, and any number of wallets can subscribe, without the watcher needing to know who they are.

The broker is implemented as a hierarchy — see the next chapter for why and how. For now: there is a root broker, regional brokers below it (gov, edu, health, etc.), and wallets at the leaves.

The wallet handler

When a wallet receives a vdr-notify message, it goes through five steps:

- **Verify the watcher's signature** against the watcher's DID document.
- **Check the subscription list** — has the user explicitly subscribed to this watcher? If not, log and drop.
- **Cross-check against the VDR independently** — read the referenced transaction directly from the ledger to confirm the watcher's claim. This is advisory, but recommended for high-stakes events like revocations.
- **Apply the change to local state** — update the credential's revocation status, refresh the cached DID document, and so on.
- **Surface to the user** if appropriate — alert them, refuse to present the affected credential next time, or just record silently in the audit log.

THE SCIENTIFIC CORE

07 A hierarchy that wakes only the right wallets

A flat broker — a single topic exchange with all wallets connected to it — works for ten wallets. It does not work for ten thousand. The scientific contribution of this project is the hierarchical broker design: a tree of brokers with topic-based routing at every tier, so that messages flow only along the branches that have matching subscribers, and only the wallets that care about a particular event are actually notified.

Why hierarchy

- **Locality of failure** — a regional broker outage affects only wallets in that region. The rest of the system continues.
- **Locality of load** — wallets connect to the nearest regional broker, not the root. The root sees only inter-region traffic.
- **Privacy as a side effect** — a wallet's subscriptions are visible to its regional broker but not to the root, and not to other regions.
- **Independent administration** — different realms (government, education, health) can run their regional brokers under different operational policies.

The topology

The watcher publishes to the root broker. The root has bindings of the form `ssi.gov.#`, `ssi.edu.#`, `ssi.health.#` that forward matching messages to each regional broker. Within a regional broker, individual wallets bind to topics matching their own DIDs — for example `ssi.gov.revoke.did:sov:5XYZ`. Only that wallet's queue receives that message.

Implementation choices

Option	Strengths	Weaknesses
RabbitMQ + federation	Mature, AMQP-standard, rich management UI, well-documented federation plugin.	Heavier; per-region instance is a separate Erlang VM.

NATS JetStream	Lightweight, native hierarchical subjects, excellent throughput, simple ops.	Younger ecosystem, fewer enterprise integrations.
Apache Kafka with MirrorMaker	Highest throughput, durable replay.	Designed for sequential logs, not pub/sub fan-out; awkward fit.

Three candidate broker stacks. The project's recommendation is RabbitMQ for the federation tooling, with NATS as a fallback if the JVM/Erlang footprint becomes limiting.

Subscription registry

Each regional broker maintains a subscription registry mapping wallet DIDs to topic bindings. When a wallet onboards, it sends a signed subscription request naming the DIDs it cares about. The registry verifies the request, creates the bindings on the broker, and returns a confirmation. Subscriptions can be added and removed at any time — the audit log tracks every change.

> **Trust model**
>
> Wallets only honor notifications from watchers they have explicitly subscribed to. The broker is treated as untrusted infrastructure — it can drop messages, delay them, or duplicate them, but it cannot forge them, because every notification is signed end-to-end by the watcher and the wallet verifies the signature before acting.

ROUTING

08 Topics, events, and how a message finds its wallet

Every message rides a topic. The topic is what makes routing efficient and what lets wallets express interest with precision.

The topic schema

```
ssi . {realm} . {event_type} . {did}
```

Each segment is meaningful:

Segment	Examples	Purpose
`ssi`	`ssi`	fixed namespace, prevents accidental binding from other systems
`realm`	`gov, edu, health`	the regional broker that handles this domain
`event_type`	`revoke, update, rotate, suspend`	what happened
`did`	`did:sov:5XYZ, did:web:example.com`	what it happened to

Five event types

Event type	What triggers it
`did-doc-updated`	a DID document's service endpoints or controllers change
`key-rotated`	a verification method on a DID is replaced
`vc-revoked`	a credential's revocation registry bit flips on
`issuer-suspended`	an issuer is removed from a trust registry the user follows

`schema-updated`	a newer version of a credential schema is published

A worked example

Alice holds a driving licence credential issued by the fictional government issuer GovCorp. The credential's DID is `did:sov:5XYZ`. Alice's wallet, on first receiving the credential, registers a subscription with its regional broker:

```
subscribe topic = ssi.gov.revoke.did:sov:5XYZ
```

Three months later, GovCorp revokes Alice's licence. They write a `REVOC_REG_ENTRY` transaction to the VON ledger. Within seconds, the watcher polls the new transaction, computes the accumulator delta, determines that credential index 142 has been revoked, looks up the corresponding DID, and publishes:

```
publish topic = ssi.gov.revoke.did:sov:5XYZ
{
  type: 'vc-revoked',
  did: 'did:sov:5XYZ',
  issuer: 'did:sov:GovCorp',
  ledger_txn: 8742,
  timestamp: 1714287622,
  sequence: 14091,
  signature: '...'
}
```

The root broker forwards to the gov regional broker. The gov broker forwards to Alice's wallet — and only to Alice's wallet, because no other wallet has bound that specific topic. Bob, who lives in the same region but never had GovCorp credentials, is not woken.

TECHNOLOGY STACK

09 Standards, not invention

Standing on what is already proven, so the new work is the broker hierarchy and the transparency layers — not yet another DID resolver.

Concern	Pick	Notes
DID method (primary)	`did:sov`	*anchored on the local VON ledger*
DID method (offline)	`did:key`	*for tests and air-gapped scenarios*
DID resolution	`indy-vdr`	*official Hyperledger client; talks to the VON pool*
Credential format	`AnonCreds`	*Indy-native, includes ZKP and predicates*
Credential format (alt)	`SD-JWT-VC`	*for did:web issuers, IETF draft*
DIDComm	`DIDComm v2 (didcomm-rust)`	*current IETF-compatible reference implementation*
Credential exchange	`Aries RFC 0453, 0454`	*issue-credential v2, present-proof v3*
OpenID flows	`OpenID4VCI / OpenID4VP`	*for web-based issuers and verifiers*
Mediator	`Cloud Agent Python`	*or aries-mediator-service self-hosted*
Broker	`RabbitMQ + federation plugin`	*topic exchange with cross-region forwarding*
Broker (alt)	`NATS JetStream`	*lighter weight, native hierarchical subjects*
Wallet agent framework	`Credo-TS`	*TypeScript, modular, actively developed*
Crypto	`libsodium, @noble/curves`	*Ed25519, X25519, and platform secure enclave*
Storage	`SQLCipher`	*encrypted SQLite, cross-platform*
UI (mobile)	`React Native`	*shared code with future web/desktop builds*

Concrete library choices. Where alternatives are listed, the project default is the first option. Adapters allow later substitution.

On Credo-TS

Credo-TS (formerly Aries Framework JavaScript) is the most actively maintained TypeScript SSI agent framework. It supports both AnonCreds and W3C VC credentials, DIDComm v1 and v2, and has a modular architecture that lets us slot in our custom vdr-notify protocol handler without touching the core. For a master's project this is the path of least resistance.

IMPLEMENTATION ROADMAP

10 Six phases, twenty weeks

A twenty-week plan from empty repository to a defendable thesis. Phase boundaries are deliberate — each phase has an exit criterion that can be demonstrated in five minutes, which keeps the project on track and gives the supervisor concrete checkpoints.

Phase 1 — Foundations (weeks 1–3)

Set up the core wallet skeleton without any external dependencies. By the end of this phase, two test agents can exchange a signed DIDComm trust-ping and the audit log records the round trip.

- Initialise the Credo-TS project; configure platform secure enclave bindings.
- Wire up the encrypted SQLCipher store with per-record DEKs wrapped by KEK in TEE.
- Implement DID resolvers for did:key and did:web.
- Stand up the agent skeleton; register with a public mediator (mediator.rootsid.cloud) for testing.
- Build the empty UX shell with three tabs: Inbox, Credentials, Activity.
- Implement the audit log primitives — append-only, hash-chained, exportable.

Phase 2 — VON network up (weeks 4–5)

Bring the local VON network online and integrate it into the wallet. By the end of this phase, the wallet can resolve did:sov DIDs from the local pool.

- Clone and start the bcgov/von-network in Docker.
- Write test schemas (passport, diploma, vaccination) using the genesis transactions.
- Add an indy-vdr resolver to the wallet's DID resolution chain.
- Verify with a smoke test: write a DID, read it back, see it in the wallet's audit log.

Phase 3 — Credential exchange (weeks 6–8)

Implement issue-credential v2 and present-proof v3 over DIDComm. By the end of this phase, a test issuer can issue an AnonCreds credential to the wallet, and a test verifier can request and receive a presentation with selective disclosure.

- Implement the issue-credential v2 protocol handler.

- Implement the present-proof v3 protocol handler with AnonCreds proof generation.
- Build the disclosure picker UI — a selectable list of credentials and attributes.
- Add the on-request transparency surface: verifier identity, fields requested, purpose, retention.

Phase 4 — Watcher and broker (weeks 9–12)

The largest phase. Build the watcher and the hierarchical broker. By the end, revoking a credential on the VON ledger causes the affected wallet — and only the affected wallet — to update its local state and alert the user.

- Build the watcher service: indy-vdr poller, transaction classifier, topic builder.
- Implement the revocation accumulator delta resolver (the genuinely hard part).
- Stand up the root broker (RabbitMQ) and three regional brokers via the federation plugin.
- Build the subscription registry with signed subscription requests.
- Implement the vdr-notify protocol handler in the wallet.
- Wire the cross-check-against-VDR step on receipt of a notification.

Phase 5 — Transparency layers (weeks 13–16)

Polish the two transparency channels. By the end, the user can replay any past interaction and export a signed receipt.

- Build the live behind-the-scenes log view, streaming events as they happen.
- Implement plain-language event templates for each event type.
- Add the audit-log replay UI with filters by date, counterparty, and event type.
- Implement signed receipt export — JSON-LD with embedded proof.
- Periodic anchoring of the audit log root hash to the VON ledger.

Phase 6 — Evaluation and write-up (weeks 17–20)

Run the experiments, analyse the results, and write the thesis. The experiments are described in the next chapter.

- Run scalability benchmarks: 10, 100, 1000 wallets across 1, 2, 3 broker tiers.
- Run privacy analysis: per-DID vs realm-wide subscriptions, measure information leakage to the broker.
- Run resilience tests: kill regional brokers, measure recovery time and message loss.
- Write up: introduction, related work, design, implementation, evaluation, conclusion.
- Defence preparation.

SCIENTIFIC CONTRIBUTION

11 What this contributes, and how we'll measure it

A master's thesis needs a question, not just an implementation. This project's questions, and the experimental approach for answering them.

Three research questions

RQ1 — Scalability

How does notification latency grow with the number of wallets and the depth of the broker hierarchy? A flat broker presumably handles small loads well; the hypothesis is that a two- or three-tier hierarchy with topic-based routing scales linearly with the number of wallets per region while keeping cross-region traffic bounded.

RQ2 — Privacy

What is the privacy cost of fine-grained topics versus broad ones? Per-DID topics give precise routing but reveal each wallet's DID interests to its regional broker. Realm-wide topics hide interest but wake every wallet in the realm. There is a trade-off; the project will characterise it quantitatively.

RQ3 — Resilience

How does the system behave under partial broker failure? When a regional broker is killed, what is the message loss, what is the recovery time after restart, and what does the wallet user experience look like during the outage?

Experimental setup

All experiments run on a single workstation using Docker Compose to orchestrate the watcher, brokers, mediator, and simulated wallet population. Wallets are implemented as headless Credo-TS instances that subscribe to topics and record delivery timestamps. The VON network and the wallet population can be reset between runs for clean baselines.

Metrics

Metric	What it measures	How we'll plot it

p50 / p95 latency	time from watcher publish to wallet receipt	as a function of wallet count and broker depth
Wakeup precision	ratio of relevant to total notifications received	as a function of topic granularity
Subscription leakage	what the broker can infer about a wallet's interests	qualitative, with a worked attack example
Recovery time	time to drain backlog after broker restart	as a function of outage duration
Memory footprint	broker memory growth with subscription count	as a function of wallets and topics per wallet

> **The privacy frontier**
>
> The most interesting result will be a plot with topic granularity on one axis and two lines: information leakage to the broker, and unnecessary wakeups received by wallets. As topic granularity increases, leakage goes up and unnecessary wakeups go down. The shape of the curve, and the location of any practical sweet spot, is the project's empirical contribution.

RISKS AND MITIGATIONS

12 Five honest concerns and how we'd handle them

Every system has soft spots. Naming them early, in a public document, is the honest way to design — and a defendable thesis explicitly addresses each.

#	Risk	Mitigation
1	**A single watcher can lie**	*Allow multiple watchers per user; cross-check before acting on notifications. Treat watcher claims as advisory; the ledger is the source of truth.*
2	**Subscriptions leak interest to the broker**	*Allow broader topic bindings (realm-wide) for sensitive cases at the cost of more wakeups; offer polling-only mode as a fallback for the most sensitive users.*
3	**Stale notify messages get replayed**	*Sequence numbers per (watcher, wallet) pair; signed timestamps; idempotent handlers that detect duplicates.*
4	**A broker outage drops messages**	*Durable queues at every tier; periodic background polling against the ledger as a fallback; multiple regional broker replicas.*
5	**The audit log is tampered with locally**	*Hash chain across log entries plus periodic anchoring of the root hash to the VON ledger. The user can prove the log's integrity to any third party.*
6	**The mediator becomes a privacy chokepoint**	*Allow the user to switch mediators or run their own; do not embed sensitive data in wakeup notifications.*
7	**Credo-TS or didcomm-rust changes break the agent**	*Pin versions; maintain an upgrade test harness; budget time in phase 6 for one major dependency upgrade.*

CLOSING THOUGHTS

13 What we are building toward

A wallet that earns trust by showing its work.

This document has described, in increasing detail, what that means: a four-layer wallet architecture; two distinct transparency channels; a local VON ledger as the trust anchor; a watcher service that converts pull-based ledger semantics into a push channel; a hierarchical broker that scales the push channel by routing only to interested wallets; a six-phase, twenty-week plan to implement and evaluate it.

What makes this a master's project rather than just an engineering exercise is the scientific framing: three research questions about scalability, privacy, and resilience, with concrete experiments planned and metrics defined. The project's expected contribution is a quantitative characterisation of the privacy-versus-efficiency trade-off in topic-routed SSI notification systems — an area where, to our knowledge, no published evaluation currently exists.

Three things to do first

Stand up VON. A single command brings up the local Indy ledger. Write a test schema. See a transaction in the browser. The project is suddenly real.

Pick the framework. Credo-TS is the recommendation. Clone the starter, wire it to the VON pool, exchange a trust-ping between two test agents.

Plan the experiment. Decide before any wallet code is written: what is the primary scalability metric, what is the wallet count to which we want to scale, what is the topology to be benchmarked. The experimental design constrains the implementation, and that is a feature, not a bug.

A transparent wallet is not a UI feature. It is the architecture choice that makes the UI possible.

Thank you.